These two methods—matches? and failure_message—are all you need to define a simple matcher. The protocol contains several optional methods you can use to customize your matcher's behavior further; we'll discuss those in *Using the Matcher DSL*, on page 220.

Composing Matchers

Even on their own, matchers are powerful tools for your tests. But they really shine when you *compose* them with other matchers to specify *exactly* what you expect (and nothing more). The result is more robust tests and fewer false failures.

There are a few different ways to compose matchers:

- Pass one matcher directly into another
- Embed matchers in Array and Hash data structures
- Combine matchers with logical and/or operators

Before we consider these three cases, let's look at how matchers determine whether or not the subject matches.

How Matchers Match Objects

Matchers build on top of one of Ruby's standard protocols in order to provide composability: the humble === method. This method, often called "three-quals" or "case equality," defines a *category* to which other objects may (or may not) belong. Ruby's built-in regular expressions, ranges, and classes define this operator, and you can add it to your own objects as well.

Let's see how case equality works in an IRB session:

```
>> /^\[warn\] / === '[warn] Disk space low'
=> true
>> /^\[warn\] / === '[error] Out of memory'
=> false
>> (1..10) === 5
=> true
>> (1..10) === 15
=> false
```

We're playing with === here just to get a feel for how Ruby (and RSpec) use it internally. Most of the time, you won't call it directly from production code. Instead, Ruby will call it for you inside each when clause of a case expression.

Just to drive home the point that matchers are plain old Ruby objects that implement ===, here's what a matcher would look like inside a case expression: